

AWS API GATEWAY

Complete Mastery Guide

Zero → Hero | Real Scenarios | Interview Ready

Covers: HTTP API • REST API • WebSocket API • JWT Auth • mTLS • Rate Limiting
CORS • Throttling • Usage Plans • VPC Links • Custom Domains • Caching & More

PART 1 — FUNDAMENTALS & MENTAL MODEL

1. What is API Gateway & Why It Exists

The Problem Without API Gateway

In a microservices architecture, clients (mobile apps, web browsers, third-party partners) need to talk to dozens of backend services. Without a central gateway, every client connects directly to every service:

- No central authentication — every service must implement its own auth logic
- No throttling — any client can flood any service
- Tight coupling — clients break when service URLs change
- No observability — difficult to log and monitor cross-service calls
- No versioning — breaking changes impact all consumers immediately

The Solution: API Gateway as a "Programmable Front Door"

API Gateway is a fully managed service that sits between clients and your backend. Think of it as a smart security guard + receptionist + traffic controller all in one.

Concern	Without Gateway	With API Gateway
Authentication	Each service handles it	Enforced centrally before traffic reaches backend
Rate limiting	None or per-service	Configurable per account/stage/route/key
Client coupling	Hardcoded service URLs	Clients call single stable endpoint
Observability	Fragmented logs	Centralized CloudWatch logs + metrics
Traffic control	None	Throttling, burst limits, usage plans
Protocol translation	Manual mapping	HTTP, WebSocket, AWS service integrations

💡 Core Mental Model

API Gateway = policy layer + control layer. It does NOT execute business logic. Keep all business logic in your backend (Lambda, ECS, EKS, EC2). The gateway handles: auth, routing, throttling, transformation, observability.

Real-World Architecture Placement

In a production system, API Gateway sits as the entry point, often layered with other services:

```
Client (Browser / Mobile / Partner)
```

```
↓
CloudFront (CDN — caching, edge delivery, DDoS mitigation)
↓
AWS WAF (Web Application Firewall — OWASP rules, IP blocklists)
↓
Amazon API Gateway (auth, throttling, routing, transformation)
↓
ALB / NLB (load balancing, connection reuse for high-throughput)
↓
Backend: Lambda / EKS / ECS / EC2
↓
Data Layer: RDS / DynamoDB / ElastiCache
```

Interview Trap

Interviewers will ask: "Is API Gateway a load balancer?" The correct answer is NO. A load balancer (ALB) distributes traffic across targets. API Gateway is an API management layer — it handles auth, throttling, transformation, and routing. Use BOTH together in real production systems.

2. Types of API Gateway in AWS

AWS offers three distinct API Gateway types. Choosing the wrong one is a common and costly mistake.

Type	Best For	Key Features	Pricing
HTTP API	Most APIs (80% of use cases)	JWT auth, CORS, Lambda proxy, low latency	~\$1/million requests
REST API	Advanced enterprise features	API keys, usage plans, request validation, mapping templates, caching, WAF integration	~\$3.50/million requests
WebSocket API	Real-time bidirectional apps	Persistent connections, chat, live dashboards, gaming	~\$1/million messages

When to Use Each Type — Real Scenarios

HTTP API — Use This 80% of the Time

- Mobile app backend (user authentication via Cognito JWT, CRUD operations)
- Startup MVP — fast setup, low cost, Lambda integration
- Microservice routing — /users → Lambda A, /orders → Lambda B
- Any API where you need JWT validation and nothing more advanced

REST API — Use When You Need Advanced Control

- Partner API monetization — API keys + tiered usage plans (free vs paid tiers)

- Legacy backend integration with request/response transformation (mapping templates)
- APIs that require request body validation against a JSON schema
- APIs that need response caching to reduce Lambda/backend invocations
- WAF integration at the API Gateway level

WebSocket API — Real-Time Use Cases

- Chat applications (Slack-style messaging)
- Live sports scores / stock tickers
- Multiplayer gaming — real-time state sync
- Collaborative document editing (Google Docs-style)
- IoT device telemetry dashboards

Decision Rule

Start with HTTP API. Upgrade to REST API only when you need: API keys + usage plans, request body validation with JSON Schema, mapping templates, or response caching.

PART 2 — CORE CONCEPTS IN DEPTH

3. Building Blocks — Routes, Integrations, Stages

Routes (Path-Based Routing)

A route maps an HTTP method + path to an integration (backend). This is the foundation of how traffic flows through the gateway.

Route	Integration Target	Real-World Use
GET /users	Lambda: getUsersFunction	Fetch list of users from DynamoDB
POST /orders	ALB → EKS service	Create new order, complex business logic in container
POST /payments	HTTP endpoint (EC2)	Process payment via legacy payment service
POST /notify	AWS Service: SQS	Queue notification — async, decoupled
GET /products	Lambda (cached)	Product catalog — cache 60s to reduce DB load
ANY /{proxy+}	Lambda: catch-all	Proxy all unmatched routes to one Lambda handler

URL Structure: Stage + Route = Final Endpoint

This is where many engineers get confused. The full URL is always: Stage URL + Route Path.

```
Stage: $default → https://abc123.execute-api.ap-south-1.amazonaws.com
Stage: dev      → https://abc123.execute-api.ap-south-1.amazonaws.com/dev
Stage: prod     → https://abc123.execute-api.ap-south-1.amazonaws.com/prod
```

```
To call GET /users in dev stage:
https://abc123.execute-api.ap-south-1.amazonaws.com/dev/users
```

```
To call GET /users in $default stage:
https://abc123.execute-api.ap-south-1.amazonaws.com/users
```

⚠ Common Mistake

Calling `https://.../users` when you expect the dev stage will actually hit `$default`. Always include the stage prefix for non-default stages. Missing this causes silent routing issues in multi-stage setups.

Stages — Environments & Deployment Lifecycle

Stages map to your deployment environments. They control not just the URL prefix, but also deployment versions, throttling configs, logging, and variable injection.

Stage	Auto Deploy	URL Prefix	Use Case
\$default	Enabled	None (root)	Quick prototyping — changes go live immediately. Never use in production.
dev	Disabled	/dev	Development testing. You control exactly when to deploy.
qa	Disabled	/qa	QA/staging testing before production release.
prod	Disabled	/prod	Live production traffic. Controlled, audited releases only.

Stage Variables

Stage variables are key-value pairs that act like environment variables for your stage. This allows one API definition to behave differently across environments.

```
Stage Variable: lambdaAlias = DEV
Integration URI:
arn:aws:lambda:region:account:function:myFunc:${stageVariables.lambdaAlias}
```

- Dev stage → lambdaAlias = DEV → invokes myFunc:DEV alias
- Prod stage → lambdaAlias = PROD → invokes myFunc:PROD alias
- No need to change the API definition — just change the stage variable

Integration Types

Integration Type	When to Use	Notes
Lambda Proxy (AWS_PROXY)	Serverless APIs	Gateway passes raw event to Lambda. Lambda controls response format.
HTTP Proxy	External HTTP APIs, legacy backends	Gateway forwards request as-is. Minimal transformation.
HTTP (Custom)	When you need request/response mapping	Use with mapping templates to transform payloads.
AWS Service	Direct SQS, SNS, Step Functions, DynamoDB	No Lambda needed. Reduces latency and cost for simple integrations.
Mock	Testing, returning static responses	Gateway returns response without calling any backend.
VPC Link	Private ALB/NLB inside VPC	Required when backend is not publicly accessible.

4. Authentication & Authorization — In Depth

API Gateway supports multiple authentication mechanisms. Understanding when to use each is critical for both security and interview performance.

Option 1: JWT Authorizer (Most Common for HTTP API)

JWT (JSON Web Token) is the standard for stateless authentication. The gateway validates the token signature before any backend is invoked.

JWT Structure

```
Format: HEADER . PAYLOAD . SIGNATURE
```

```
Header: { "alg": "RS256", "typ": "JWT" }
Payload: { "userId": 123, "role": "admin", "exp": 1720000000, "iss":
"https://cognito.amazonaws.com/pool" }
Signature: RS256(base64(header) + "." + base64(payload), privateKey)
```

Key Insight

JWT payload is base64 encoded, NOT encrypted. Anyone can decode and read the claims. Never put sensitive data (passwords, SSNs, secrets) in a JWT payload. Security comes from the cryptographic signature, not from hiding the data.

JWT Authentication Flow — Step by Step

- 1. User submits credentials to auth server (Cognito, Auth0, custom)
- 2. Auth server validates and returns JWT (access token + refresh token)
- 3. Client stores JWT securely (HttpOnly cookie or secure storage)
- 4. Client sends JWT in every API request: Authorization: Bearer <token>
- 5. API Gateway validates: signature, expiry (exp claim), issuer (iss), audience (aud)
- 6. If valid: Gateway forwards request + decoded claims to backend
- 7. Backend reads user identity from forwarded claims — NO DB lookup needed

Configuring JWT Authorizer

```
# AWS CLI — Create JWT Authorizer on HTTP API
aws apigatewayv2 create-authorizer \
  --api-id abcl23 \
  --authorizer-type JWT \
  --identity-source '$request.header.Authorization' \
  --jwt-configuration '{
    "Audience": ["myapp-client-id"],
    "Issuer": "https://cognito-idp.ap-south-1.amazonaws.com/ap-south-1_POOLID"
  }' \
  --name cognitoAuthorizer
```

JWT Token Types (Important for Real Systems)

Token Type	Lifetime	Purpose	Stored Where
Access Token	5–60 minutes	API authorization — sent with every API call	Memory (JS) or HttpOnly cookie
Refresh Token	Days to months	Obtain new access token when expired	HttpOnly cookie only
ID Token	5–60 minutes	User identity info for frontend display	Memory (JS)

Option 2: Lambda Authorizer

Lambda authorizer gives you full custom control. The gateway calls your Lambda with request data, and your Lambda decides allow/deny. Use this when JWT alone isn't enough.

Real-World Use Cases for Lambda Authorizer

- Validate custom session tokens (non-standard format)
- Check API key against your own database (SaaS licensing validation)
- Implement attribute-based access control (ABAC) — e.g., tenant isolation
- Rate limiting per user with custom logic in Redis/DynamoDB
- Validate custom headers or query parameters beyond JWT

```
# Lambda Authorizer response (IAM policy)
{
  "principalId": "user-123",
  "policyDocument": {
    "Version": "2012-10-17",
    "Statement": [{
      "Action": "execute-api:Invoke",
      "Effect": "Allow",    // or "Deny"
      "Resource": "arn:aws:execute-api:region:account:api/stage/method/resource"
    }]
  },
  "context": { "userId": "123", "role": "admin", "tenantId": "acme" }
}
```

Option 3: IAM Authorization

Use IAM auth for internal AWS-to-AWS calls. The calling service must sign requests with AWS Signature Version 4 (SigV4).

- Microservice A (in EKS) calling internal API Gateway endpoint
- AWS Step Functions calling an internal API
- CI/CD pipeline triggering API operations

 **Don't Use IAM Auth for**

Browser or mobile clients. IAM signing from a browser would expose AWS credentials. Use JWT (Cognito) for all end-user authentication.

JWT Revocation — The Hard Problem

JWT is stateless — once issued, it's valid until expiry. This creates a challenge: how do you immediately invalidate a token (e.g., user logs out, account suspended)?

Strategy	How It Works	Trade-off
Short expiry (5–15 min)	Token expires quickly; damage window is small	User must refresh frequently
Token blacklist (Redis)	Store revoked JTI (token ID) in Redis; check on each request	Adds latency; partial statefulness
Secret rotation	Change signing key; all existing tokens instantly invalid	Affects ALL users, not just one
Refresh token revocation	Invalidate refresh token; user must re-login next refresh cycle	Doesn't help for current access token lifetime

5. Throttling — Protecting Your System

Throttling is one of the most critical API Gateway features. Without it, a single bad actor or traffic spike can crash your backend and generate massive costs.

Three Levels of Throttling

Level	Scope	Config Location	Use Case
Account-level	All APIs in the AWS region	Service Quotas console	Hard ceiling. AWS sets default (2500 rps). Request increase for high-volume APIs.
Stage-level (Default Route)	All routes in a stage except those with custom settings	Stage → Default route throttling	Different limits per environment (dev: 50 rps, prod: 2000 rps).
Route-level	Specific route only, overrides stage default	Stage → Route settings	Protect sensitive endpoints like POST /login from brute force.

Rate Limit vs Burst Limit — The Highway Analogy

These two settings work together to shape traffic:

Setting	What It Controls	Example	Analogy
Rate (rps)	Sustained requests per second, long-term average	100 rps	Normal highway speed limit — steady flow
Burst	Peak requests allowed in a token-bucket spike	500	Passing lane — short acceleration allowed, then back to limit

How the Token Bucket Algorithm Works

```
Bucket capacity = Burst limit (e.g., 500 tokens)
Token refill rate = Rate limit (e.g., 100 tokens/sec)

Request arrives → consume 1 token from bucket
Bucket empty → request throttled → 429 Too Many Requests returned
Tokens refill continuously at rate limit speed
```

Real-World Throttling Configuration Examples

Endpoint	Rate (rps)	Burst	Reason
POST /login	5	20	Brute force protection — attacker trying thousands of passwords per second

Endpoint	Rate (rps)	Burst	Reason
GET /products	500	1000	High-read endpoint — product listings hit frequently by all users
POST /payment	50	100	Backend payment processor has rate limits; protect from accidental floods
POST /register	10	30	Abuse prevention — stop bot account creation
GET /health	1000	2000	Health checks — needs high availability, no business risk

Production Best Practice

Always set route-level throttling on authentication endpoints (login, password reset, OTP verification). These are the highest-value targets for abuse. Even if your account limit is 10,000 rps, a brute force attack on /login should be stopped at 5-10 rps.

6. API Keys & Usage Plans — Monetization & Client Control

API Keys combined with Usage Plans allow you to differentiate service levels for different client applications. This is essential for SaaS API products and partner APIs.

The Relationship Model

```

API Key (identifies a client/app)
  ↓ attached to
Usage Plan (defines rate + quota limits for that key)
  ↓ associated with
API Stage (which API and environment the plan applies to)

```

What Each Component Does

Component	Purpose	Example
API Key	Identifies the calling client application (not user)	x-api-key: abc123xyz — Company A's integration key
Usage Plan — Rate	Per-second request rate for clients on this plan	Free plan: 10 rps Premium plan: 200 rps
Usage Plan — Burst	Temporary spike tolerance	Free plan: 50 Premium plan: 400
Usage Plan — Quota	Maximum requests per day/week/month	Free: 1,000/day Pro: 100,000/day Enterprise: unlimited

Component	Purpose	Example
Throttling	Global system protection (no key needed)	Account: 5,000 rps — protects backend regardless of keys

API Keys vs Throttling — Critical Difference

Feature	Throttling	API Keys + Usage Plans
Scope	Global — applies to all traffic	Per-client — each key has its own limits
Requires API key	No	Yes — key must be present in x-api-key header
Purpose	Protect backend from overload	Control and monetize individual clients
Quota support	No	Yes — daily/weekly/monthly request caps
Available in	HTTP API + REST API	REST API only

Combined Behavior — Both Apply Simultaneously

Example: Account throttle = 10,000 rps | Usage plan for Client A = 200 rps
→ Client A is limited to $\min(10,000, 200) = 200$ rps
→ Even if Client A upgrades plan to 500 rps, account limit is still the ceiling

Real-World Scenario: SaaS API Product

Plan Tier	API Key	Rate (rps)	Daily Quota	Price
Free	Key A	10	1,000 requests	\$0/month
Starter	Key B	50	10,000 requests	\$29/month
Professional	Key C	200	100,000 requests	\$149/month
Enterprise	Key D	1,000	Unlimited	Custom pricing

Critical: API Keys ≠ Security

API keys identify and track clients. They are NOT a security mechanism. A key can be copied from client-side code, logs, or network captures. Always combine API keys with JWT auth or mTLS for actual security. The key's value is rate limiting and usage tracking, not authentication.

PART 3 — SECURITY, NETWORKING & ADVANCED FEATURES

7. mTLS — Mutual TLS for B2B Secure APIs

Standard HTTPS is one-way TLS — the server proves its identity to the client. Mutual TLS (mTLS) requires BOTH sides to prove identity with certificates. This is the gold standard for B2B integrations.

mTLS vs Standard TLS

Aspect	Standard TLS (HTTPS)	Mutual TLS (mTLS)
Server proves identity	✓ Yes	✓ Yes
Client proves identity	✗ No	✓ Yes — via client certificate
Protection against	Eavesdropping, MITM	All of TLS + unauthorized clients
Credential type	Password/Token	X.509 certificate
Use case	Public APIs (mobile, web)	B2B integrations, financial APIs, healthcare

mTLS Implementation in API Gateway

Step 1: Create a Certificate Authority (CA) or use your existing one

```
# Generate CA private key and self-signed cert
openssl genrsa -out ca.key 4096
openssl req -new -x509 -days 3650 -key ca.key -out ca.crt -subj '/CN=MyAPI-CA'
```

Step 2: Issue client certificate to your partner

```
openssl genrsa -out partner.key 2048
openssl req -new -key partner.key -out partner.csr -subj '/CN=PartnerCompany'
openssl x509 -req -days 365 -in partner.csr -CA ca.crt -CAkey ca.key -out partner.crt
```

Step 3: Upload truststore to S3 and enable mTLS on custom domain

```
# Upload the CA certificate (truststore) to S3
aws s3 cp ca.crt s3://my-truststore-bucket/truststore.pem

# Enable mTLS on custom domain
aws apigatewayv2 create-domain-name \
  --domain-name api.partner.mycompany.com \
  --domain-name-configurations 'CertificateArn=arn:aws:acm:... ' \
  --mutual-tls-authentication 'TruststoreUri=s3://my-truststore-bucket/truststore.pem'
```

mTLS Real-World Scenario: Financial API

A banking API that exposes credit scoring data to licensed fintech partners uses mTLS as the primary authentication mechanism. Each partner receives a unique client certificate signed by the bank's CA. If a partner's certificate expires or is compromised, the bank revokes it by updating the truststore — that partner immediately loses access without needing to rotate any passwords or API keys.

8. CORS — Cross-Origin Resource Sharing

CORS is a browser security feature that prevents web pages from making requests to a different domain than the one that served the page. API Gateway must be configured to allow your frontend's origin.

Why CORS Errors Happen

```
Frontend served from: http://localhost:3000 (development)
                      OR https://app.mycompany.com (production)
API at:               https://api.mycompany.com
→ Browser blocks the request because origins differ (different domain/port)
```

How CORS Works — Preflight Request

- Browser sends OPTIONS preflight request to API before the actual request
- API Gateway responds with CORS headers indicating what's allowed
- Browser reads response — if origin is allowed, sends actual request
- API Gateway adds CORS headers to actual response too

CORS Configuration in API Gateway

Setting	Value Example	Meaning
Allow-Origin	https://app.mycompany.com	Only this origin can call the API
Allow-Methods	GET,POST,PUT,DELETE,OPTIONS	Which HTTP methods are permitted
Allow-Headers	Content-Type,Authorization,X-API-Key	Which request headers are allowed
Max-Age	86400 (seconds)	Browser caches preflight result for 24 hours — fewer OPTIONS calls
Allow-Credentials	true	Allows cookies and auth headers (cannot use * for origin with this)

CORS Configuration Examples

```
# AWS CLI — Enable CORS on HTTP API
aws apigatewayv2 update-api \
  --api-id abc123 \
```

```
--cors-configuration '{
  "AllowOrigins": ["https://app.mycompany.com"],
  "AllowMethods": ["GET", "POST", "PUT", "DELETE", "OPTIONS"],
  "AllowHeaders": ["Content-Type", "Authorization"],
  "MaxAge": 86400,
  "AllowCredentials": true
}'
```

Environment	Allow-Origin Setting	Why
Development	http://localhost:3000	Local frontend URL only
Staging	https://staging.app.mycompany.com	Staging frontend only
Production	https://app.mycompany.com	Exact production domain — no wildcards
Public API (non-credentialed)	* (wildcard)	Any origin — only when no cookies or auth headers needed

Production CORS Mistake

NEVER use Allow-Origin: * in production APIs that handle user data or auth tokens. Wildcard CORS allows any website to make authenticated calls to your API if the user has an active session. Always specify exact origins in production.

9. Custom Domains & SSL/TLS Termination

Instead of exposing the auto-generated `execute-api.amazonaws.com` URL to clients, map your API to a branded domain. SSL is terminated at API Gateway, not at your backend.

Setup Steps

- Step 1: Request or import a certificate in AWS Certificate Manager (ACM) — must be in us-east-1 for edge-optimized, or same region for regional
- Step 2: Create custom domain in API Gateway with the ACM certificate ARN
- Step 3: Create API mapping — map your API + stage to a path on the domain
- Step 4: Create DNS record in Route 53 (or your DNS provider) pointing to the API Gateway endpoint

```
api.mycompany.com → (Route 53 CNAME) → d-abc123.execute-api.ap-south-1.amazonaws.com
api.mycompany.com/v1 → prod stage of your API
api.mycompany.com/v2 → prod stage of your v2 API
dev.api.mycompany.com → dev stage of your API
```

SSL Termination — What It Means

When you attach an ACM certificate to API Gateway, TLS is terminated at the gateway. The gateway decrypts the HTTPS traffic, processes it, then forwards to your backend over HTTP (within the VPC) or re-encrypts depending on configuration. Your backend never needs to handle TLS, simplifying backend certificates.

Endpoint Type	Certificate Region	Use Case
Edge-optimized	us-east-1 ONLY	Global clients — leverages CloudFront edge locations for lower latency
Regional	Same region as API	Single-region deployments or when using CloudFront separately
Private	Same region as API	APIs accessible only within VPC — no public DNS

10. Private APIs & VPC Links

Not all APIs should be publicly accessible. Private APIs are reachable only from within your VPC, while VPC Links allow API Gateway to route to private backend resources.

Private API — Internal Microservices

A Private API Gateway endpoint is only accessible through a VPC Interface Endpoint (PrivateLink). No public internet access, even with a valid URL.

- Use case: Internal service-to-service communication within EKS cluster or across VPCs
- Use case: Enterprise internal tools that must never be exposed to the internet
- Use case: Compliance requirements mandating no internet egress/ingress for service calls

```
Internal App (EKS Pod)
  ↓
VPC Interface Endpoint (PrivateLink)
  ↓
Private API Gateway
  ↓
Lambda / DynamoDB / Internal Service
```

VPC Link — Connect to Private Backends

VPC Link allows a public API Gateway to route traffic to resources inside your VPC without making those resources public. The VPC Link creates an ENI (Elastic Network Interface) in your VPC.

```
Internet → API Gateway (public) → VPC Link → Private ALB → EKS services
```

Scenario	Solution	Why
Backend is public Lambda	Direct Lambda integration	Lambda has a public ARN, no VPC Link needed
Backend is public ALB/EC2	HTTP integration with public URL	Backend is already accessible
Backend is private ALB in VPC	VPC Link → Private ALB	ALB has no public IP; VPC Link bridges the gap
Backend is EKS/ECS in private subnet	VPC Link → private ALB/NLB → service	Services have private IPs only

PART 4 — ADVANCED FEATURES & OPTIMIZATIONS

11. Request & Response Transformation

Sometimes the format your client sends doesn't match what your backend expects. Transformation (supported in REST API via Mapping Templates, or via Lambda proxy logic) solves the impedance mismatch.

When Transformation Is Needed

Scenario	Problem	Solution
Legacy backend integration	Legacy SOAP/XML backend; modern clients send JSON	API Gateway transforms JSON request to XML before forwarding
Field renaming	Client sends {name: 'John'}, backend expects {username: 'John'}	Mapping template remaps fields
Adding headers	Backend requires X-Correlation-ID on every request	Gateway injects header from request context
Error sanitization	Backend returns raw stack traces	Response template maps all 5xx to {error: 'Internal error'}
Stripping sensitive fields	Backend response includes internal IDs not for clients	Response template removes fields

Mapping Template Example (VTL — REST API)

```
## Request Mapping Template
## Client sends: { "name": "Tharun" }
## Backend expects: { "username": "Tharun", "role": "user", "requestId": "..."}

{
  "username": "$input.path('$.name')",
  "role": "user",
  "requestId": "$context.requestId",
  "callerIp": "$context.identity.sourceIp"
}
```

Error Response Mapping

```
## Map all 500 backend errors to a clean client response
{
  "error": "An internal error occurred.",
}
```

```
"requestId": "$context.requestId",  
"timestamp": "$context.requestTimeEpoch"  
}
```

12. Request Validation

API Gateway can validate incoming requests BEFORE invoking your backend. Invalid requests return 400 immediately — no Lambda cold start, no backend compute costs.

What Can Be Validated

- Request body — validate against a JSON Schema model (required fields, types, patterns)
- Query string parameters — validate presence and format
- Headers — validate required headers are present

JSON Schema Model Example

```
{  
  "type": "object",  
  "required": ["email", "password"],  
  "properties": {  
    "email": { "type": "string", "format": "email" },  
    "password": { "type": "string", "minLength": 8, "maxLength": 128 },  
    "rememberMe": { "type": "boolean" }  
  },  
  "additionalProperties": false  
}
```

→ Missing email field → 400 Bad Request (immediate, no Lambda invoked)

→ Password under 8 chars → 400 Bad Request (immediate, no Lambda invoked)

→ Unknown field sent → 400 Bad Request (additionalProperties: false)

13. Caching

API Gateway caching (REST API only) stores backend responses for a configurable TTL. Subsequent identical requests return the cached response without touching your backend.

Aspect	Details
Cache TTL	Default 300s (5 min). Range: 0 to 3600s (1 hour)
Cache capacity	0.5 GB to 237 GB. Larger = more entries = higher cost
Cache key	By default: HTTP method + full path. Add query params or headers to key.

Aspect	Details
Cache invalidation	Console button, or client sends Cache-Control: max-age=0 header (if permitted)
Availability	REST API only — HTTP API does not support built-in caching
Cost	Charged per hour based on cache size — check pricing before enabling

When Caching Helps vs Hurts

Use Case	Cache?	Why
GET /products (product catalog)	✓ Yes	Changes rarely, read frequently. Cache for 60-300s saves thousands of Lambda calls.
GET /users/me (current user profile)	✗ No	Personalized per user. Caching would serve one user's data to another.
POST /orders	✗ No	Write operation — never cache writes. Would cause duplicate order creation.
GET /exchange-rates	✓ Yes (short TTL)	Updated every few minutes. Cache for 30s reduces API provider costs.
GET /dashboard/realtime	✗ No	Real-time data by definition. Stale cache defeats the purpose.

14. Monitoring & Observability

API Gateway integrates with Amazon CloudWatch for logs, metrics, and alarms. Without proper observability, debugging production issues is nearly impossible.

Key Metrics to Monitor

Metric	What It Tells You	Alarm Threshold (Example)
4xxError	Client errors — bad requests, auth failures, not found	> 5% of requests (indicates API misuse or client bugs)
5xxError	Server errors — backend failures, timeouts, Lambda errors	> 1% of requests (requires immediate investigation)
Latency (P50/P95/P99)	Response time percentiles	P99 > 3000ms (backend may be degrading)
IntegrationLatency	Time backend takes to respond	Isolates whether slowness is gateway vs backend
Count	Total request volume	Sudden spikes may indicate traffic attacks or viral events
ThrottleCount	Requests rejected by throttling	High count = throttle limits too low or actual abuse
CacheHitCount / CacheMissCount	Cache effectiveness	Low hit rate = cache key misconfigured or TTL too short

CloudWatch Logs Configuration

```
# Enable access logging on a stage (HTTP API)
aws apigatewayv2 update-stage \
  --api-id abc123 \
  --stage-name prod \
  --access-log-settings '{
    "DestinationArn": "arn:aws:logs:region:account:log-group:api-access-logs",
    "Format":
    "{\"requestId\":\"$context.requestId\",\"ip\":\"$context.identity.sourceIp\",\"routeKey\"
    :\"$context.routeKey\",\"status\":\"$context.status\",\"latency\":\"$context.responseLate
    ncy\",\"integrationError\":\"$context.integrationErrorMessage\"}"
  }'
```

Debugging Guide — Production Issues

Symptom	First Check	Root Cause & Fix
Spike in 5xx errors	IntegrationLatency metric	Backend timeout or crash. Check Lambda logs, EKS pod logs, DB connection limits.

Symptom	First Check	Root Cause & Fix
Spike in 4xx errors	Access logs — which route, which status	Auth failures (401) = token issue. 400 = validation failing. 403 = IAM/authorizer denying.
High latency	IntegrationLatency vs Latency	If Integration >> Latency: backend is slow. If similar: gateway overhead (rare).
ThrottleCount rising	Rate/Burst config on stage/route	Increase limit if legitimate traffic, or investigate abuse patterns.
CORS error in browser	OPTIONS response headers	Missing/wrong Allow-Origin or missing OPTIONS route.
502 Bad Gateway	Lambda logs	Lambda function returned invalid response format for proxy integration.
504 Gateway Timeout	IntegrationLatency	Backend exceeded 29-second timeout limit. Redesign to async if >29s needed.

29-Second Hard Limit

API Gateway has a maximum integration timeout of 29 seconds. If your backend operation takes longer (large file processing, ML inference, long DB queries), you **MUST** redesign to async: API Gateway → SQS → Lambda Worker → notify via WebSocket or polling endpoint. This is a non-negotiable architectural constraint.

PART 5 — REAL-WORLD ARCHITECTURES & PATTERNS

15. Real-World Architecture Patterns

Pattern 1: Serverless API (Simplest)

```
Client → API Gateway (HTTP API + JWT) → Lambda → DynamoDB
```

- Best for: Startups, MVPs, low-to-medium traffic APIs
- Cost: Pay per request — zero cost when idle
- Scaling: Automatic — Lambda scales with load
- Limitation: Cold starts, 15-minute Lambda timeout, 10GB memory max

Pattern 2: Microservices (Production E-commerce)

```
Client
  ↓
CloudFront (CDN + DDoS protection)
  ↓
WAF (OWASP rules, IP rate limiting, geo-blocking)
  ↓
API Gateway (HTTP API + JWT auth, route-level throttling)
  ├── GET /users → Lambda:usersFunction (read-heavy, cacheable)
  ├── POST /orders → VPC Link → ALB → EKS:ordersService
  ├── POST /pay → VPC Link → ALB → EKS:paymentService
  └── POST /notify → SQS → Lambda:notificationWorker (async)
```

Pattern 3: Async System (Long-Running Operations)

When operations take more than a few seconds, switch to async. Client gets immediate acknowledgment, checks status via polling or WebSocket callback.

```
Client → POST /reports → API Gateway → SQS → Lambda:reportGenerator
                                           ↓ (minutes later)
                                           S3 (store report PDF)
                                           ↓
Client → GET /reports/{id}/status → Lambda (check DB for status)
```

- Immediate API response: { "jobId": "abc123", "status": "queued" }
- Client polls GET /reports/abc123/status until status = completed
- Client downloads from pre-signed S3 URL when complete

Pattern 4: B2B Partner API (mTLS + API Keys)

```
Partner System → (client certificate) → API Gateway (mTLS custom domain)
                  (x-api-key header) → Usage Plan (rate limited to tier)
                                      → Lambda → RDS (tenant-isolated data)
```

- mTLS: Ensures only authorized partner systems can connect
- API Key: Identifies which partner, enforces their usage tier
- JWT from partner's IdP: Identifies the specific user within the partner's system

Pattern 5: Multi-Environment with Stage Variables

Stage	Lambda Alias	DB	Throttle	Auto Deploy
dev	DEV	dev-rds-cluster	50 rps	Yes — instant feedback
qa	QA	qa-rds-cluster	200 rps	No — QA team controls
prod	PROD	prod-rds-cluster	5000 rps	No — change control process

16. Deployment Strategies with API Gateway

Blue-Green Deployment

```
Step 1: Deploy new version → prod-v2 stage
Step 2: Validate with internal QA
Step 3: Update Route 53 weighted routing: 90% → prod, 10% → prod-v2
Step 4: Monitor metrics. If healthy, shift to 100% prod-v2
Step 5: Rename prod-v2 to prod. Keep old prod as rollback for 24h
```

Canary Releases (REST API)

REST API supports native canary deployments where a percentage of production traffic is sent to the new deployment while the rest continues on the stable version.

```
# Create a deployment with canary settings
aws apigateway create-deployment \
  --rest-api-id abc123 \
  --stage-name prod \
  --canary-settings '{
    "percentTraffic": 10,
    "useStageCache": false
  }'
```

- 10% of traffic → new canary deployment
- 90% of traffic → existing stable deployment
- Monitor CloudWatch for errors in canary
- Promote canary to 100% if healthy, or rollback if issues found

API Versioning Strategies

Strategy	Example	Pros	Cons
URL path versioning	/v1/users, /v2/users	Explicit, easy to see in logs	Separate routes or APIs needed
Stage-based versioning	stage=v1, stage=v2	Maps naturally to API Gateway stages	Stage URL in all client code
Header-based versioning	API-Version: 2024-01	Clean URLs, version in header	Harder to test in browser
Custom domain versioning	api.myapp.com/v1, v2.api.myapp.com	Clean, professional	Requires custom domain setup

Production Principle

Always maintain backward compatibility. When releasing /v2, keep /v1 running until all clients migrate. Set a deprecation timeline (e.g., 6 months), notify clients via API response headers (Deprecation: date=...), then shut down after the deadline. Sudden breaking changes destroy partner trust.

PART 6 — INTERVIEW TRAPS, DECISION GUIDE & TROUBLESHOOTING

17. Common Interview Traps & Expert Answers

Trap 1: "API Gateway vs ALB — Which do you choose?"

✗ Wrong Answer

Use API Gateway for everything — it's fully managed and easier.

✓ Expert Answer

It depends on the use case. For public-facing APIs with auth, throttling, and usage plans: API Gateway. For internal high-throughput microservice traffic (millions of rps) with simple routing: ALB is cheaper and faster. In production, I use BOTH together: API Gateway handles auth + policy enforcement for external traffic; ALB handles load balancing + connection management behind the gateway. Client → API Gateway → ALB → EKS is the standard pattern.

Trap 2: "Your backend is slow (2 sec latency). Can API Gateway fix it?"

✗ Wrong Answer

Yes — enable API Gateway caching.

✓ Expert Answer

No — API Gateway is a pass-through for latency. It waits for the backend, it doesn't speed it up. Caching helps for repeated identical GET requests, but doesn't fix the root cause. The real fix: identify the bottleneck using distributed tracing (X-Ray). Usually it's DB queries (add indexing, connection pooling, read replicas), N+1 queries, or sequential calls that should be parallelized. For operations inherently >1s, switch to async (SQS + polling). API Gateway's contribution to latency is typically <10ms and is never the bottleneck.

Trap 3: "Can API Gateway handle 1 million requests per second?"

✗ Wrong Answer

Yes, it auto-scales.

✓ Expert Answer

Not at that scale by default. Account-level throttle starts at 10,000 rps. You can request quota increases, but at 1M rps, API Gateway adds unnecessary cost and latency. At that scale, put CloudFront in front (it handles millions rps natively), bypass API Gateway for static/cacheable

content, and use ALB directly for the highest-volume backend calls. Reserve API Gateway for external API management concerns.

Trap 4: "How do you handle file uploads?"

✗ Wrong Answer

Upload through API Gateway.

✓ Expert Answer

Never upload through API Gateway for files >10MB — it has a payload size limit. The correct pattern: (1) Client requests a pre-signed S3 URL from your API. (2) API returns a time-limited S3 pre-signed URL (valid for 15 minutes). (3) Client uploads directly to S3 — bypassing API Gateway entirely. This removes the size limit, reduces Gateway cost, and offloads bandwidth from your backend.

Trap 5: "How do you secure APIs?"

✗ Weak Answer

Use API Gateway with HTTPS.

✓ Expert Answer (Defense in Depth)

Security requires multiple layers: (1) WAF: Block OWASP Top 10, SQL injection, XSS, rate-limit by IP before traffic reaches Gateway. (2) API Gateway: JWT validation (Cognito/Auth0), throttling per route, CORS restrictions. (3) Lambda Authorizer: Custom business rules (tenant isolation, feature flags). (4) Backend: Validate all input again — never trust gateway validation alone. (5) Network: Private subnets, security groups, VPC endpoints. For B2B: add mTLS. For partner monetization: add API keys + usage plans.

Trap 6: "Does API Gateway retry failed requests?"

✗ Wrong Answer

Yes, it automatically retries.

✓ Expert Answer

API Gateway does NOT automatically retry. It returns the error to the client immediately. Retry logic must be implemented at the appropriate layer: (1) Lambda: has built-in retry for async invocations (2 retries by default). (2) SQS: message remains in queue and retries based on visibility timeout + redrive policy. (3) Client: implement exponential backoff with jitter. (4) Service Mesh (Istio): can retry at the infrastructure level for synchronous microservice calls.

18. When NOT to Use API Gateway

API Gateway adds value in specific scenarios. Using it where it doesn't fit adds cost, latency, and complexity.

Use Case	Don't Use API Gateway	Use This Instead
Real-time streaming (IoT, video)	Too high latency, connection overhead	AWS IoT Core, Kinesis Data Streams, Kafka
Internal service-to-service calls at high rate	Adds cost and latency for internal calls	Service mesh (Istio/AppMesh), gRPC, ALB
Large file transfers (>10MB)	Payload size limit	Pre-signed S3 URLs, S3 Transfer Acceleration
WebSocket at massive scale	Consider cost	AWS IoT Core for device comms, custom WebSocket servers on EKS
Simple load balancing only	Unnecessary cost and complexity	ALB — cheaper, higher throughput, lower latency
Message queuing / pub-sub	Not designed for this	SQS, SNS, EventBridge, MSK (Kafka)

19. Troubleshooting Checklist

Problem	Check This	Fix
403 Forbidden on every request	Authorization header format, JWT claims (aud, iss, exp)	Ensure Bearer prefix; verify Cognito pool ID in issuer claim
CORS error in browser console	OPTIONS response headers, Allow-Origin value	Add exact frontend origin to Allow-Origins; ensure OPTIONS route exists
Route returns 404	Route definition, stage deployment status	Verify route exists and has been deployed to the target stage
502 Bad Gateway from Lambda	Lambda response format	Lambda must return {statusCode, headers, body} for proxy integration
504 Gateway Timeout	Backend processing time, IntegrationLatency metric	Optimize backend. If >29s, redesign to async with SQS.
429 Too Many Requests	Throttle config at account/stage/route level	Increase rate limit or add SQS buffer for spiky workloads
mTLS 403 errors	Client certificate CN, truststore in S3	Verify cert is signed by CA in truststore; check cert expiry
Cache not working	Cache TTL, cache key config, stage cache enabled flag	Enable cache on stage, set TTL > 0, check cache key includes relevant params
High 5xx from specific route only	That route's integration config, backend logs	Usually backend logic error — check CloudWatch logs for that function/service
VPC Link 500 errors	Security group rules, ALB health checks	Ensure API Gateway can reach ALB; check target group health

20. Production Best Practices Checklist

Category	Best Practice
Security	Enable WAF in front of API Gateway for all public APIs
Security	Use JWT authorizer — never API keys alone for authentication
Security	Set CORS to exact origin values — never wildcard in production
Security	Enable mTLS for B2B partner APIs and financial/healthcare integrations
Security	Rotate API keys periodically; immediately on suspected compromise
Reliability	Set throttling at account, stage, AND route levels — defense in depth
Reliability	Enable CloudWatch alarms on 4xxError, 5xxError, and Latency (P99)

Category	Best Practice
Reliability	Use SQS buffer for any endpoint subject to traffic spikes
Reliability	Test timeout behavior — ensure graceful degradation when backend is slow
Operations	Use separate stages for dev/qa/prod — never share stage across environments
Operations	Disable auto-deploy on prod stage — all production changes need approval
Operations	Enable access logging with structured JSON format — easier querying in CloudWatch Insights
Operations	Use stage variables to parameterize Lambda aliases, backend URLs, DB connections
Operations	Tag all API Gateway resources (API, stage, domain) with environment, team, cost-center
Cost	Use HTTP API over REST API unless you need REST-specific features
Cost	Enable caching on high-read, low-change endpoints — reduces Lambda/backend costs significantly
Cost	Monitor ThrottleCount — unexpectedly high = abuse or misconfigured clients generating cost
Developer Experience	Use custom domains — never ship raw execute-api.amazonaws.com URLs to partners
Developer Experience	Generate and maintain an OpenAPI spec — enables SDK generation and documentation
Developer Experience	Use usage plans to communicate tier limits clearly to API consumers

PART 7 — QUICK REFERENCE CARD

21. Feature → Gateway Type Matrix

Feature	HTTP API	REST API	WebSocket API
JWT Authorizer	✓	✓ (via Lambda)	✓
Lambda Authorizer	✓	✓	✓
IAM Auth	✓	✓	✓
API Keys + Usage Plans	✗	✓	✗
Request body validation	✗	✓	✗
Mapping Templates (VTL)	✗	✓	✗
Response caching	✗	✓	✗
WAF integration (direct)	✗	✓	✗
CORS configuration	✓ (built-in)	Manual	N/A
Canary deployments	✗	✓	✗
mTLS	✓ (custom domain)	✓ (custom domain)	✗
VPC Link	✓ (NLB/ALB)	✓ (NLB)	✓ (NLB)
Private API	✗	✓	✗
Price (per 1M requests)	~\$1	~\$3.50	~\$1 + \$1.15/M messages
Latency overhead	Low (~5ms)	Medium (~10ms)	Connection-based

22. Key AWS CLI Reference

Create HTTP API

```
aws apigatewayv2 create-api \  
  --name my-api --protocol-type HTTP
```

Create Route

```
aws apigatewayv2 create-route \  
  --api-id abc123 --route-key 'GET /users'
```

Create Stage

```
aws apigatewayv2 create-stage \  
  --api-id abc123 --stage-name prod
```

```
--api-id abcl23 --stage-name prod \  
--auto-deploy false
```

Deploy to Stage

```
aws apigatewayv2 create-deployment \  
--api-id abcl23 --stage-name prod
```

Set Stage Throttling

```
aws apigatewayv2 update-stage \  
--api-id abcl23 --stage-name prod \  
--default-route-settings '{"ThrottlingRateLimit": 1000, "ThrottlingBurstLimit": 2000}'
```

Create Lambda Authorizer

```
aws apigatewayv2 create-authorizer \  
--api-id abcl23 --name myAuth \  
--authorizer-type REQUEST \  
--authorizer-uri arn:aws:apigateway:region:lambda:path/.../myAuthFn/invocations \  
--identity-source '$request.header.Authorization'
```

23. Mental Models to Remember

Mental Model	What It Means
API Gateway = "Security guard + receptionist"	It controls who enters and directs them, but doesn't do the actual work inside
Throttling = "Highway speed limit"	Rate = sustained speed. Burst = passing lane — temporary acceleration allowed
JWT = "Movie ticket"	Contains your seat info, signed by the theater, verified without calling a database
Stage = "Environment"	dev/qa/prod with different configs, different backend targets, different throttle limits
API Keys = "Client badge"	Identifies the client app, enables usage tracking and per-client limits. NOT authentication.
VPC Link = "Private tunnel"	Connects public API Gateway to private resources inside your VPC without exposing them
mTLS = "Two-way handshake"	Both server AND client prove identity with certificates — highest assurance B2B security
API Gateway vs ALB	Gateway = smart policy brain. ALB = fast traffic cop. Use both together in production.